SkillSeal

Cryptographic Trust for LLM Agent Skills and Plugins

Ian McCutcheon

esoup.net

February 2026

v0.1.0

Contents

1 Abstract 5
2 Introduction 6
2.1 The Unsigned Executable Content Problem 6
2.2 The Scale of the Problem 6
2.3 Core Thesis 7
2.4 Scope 8
3 Threat Model 8
3.1 Attack Vectors and Defenses 9
3.1.1 Skill Forgery 9
3.1.2 Post-Publication Tampering 9
3.1.3 Key Substitution 9
3.1.4 Trust Store Poisoning 9
3.1.5 LLM Social Engineering 10
3.1.6 Man-in-the-Middle Key Fetch 10
3.2 What SkillSeal Does Not Prevent 10
4 Architecture 11
4.1 Design Principles 11
4.2 Components 12
4.3 Skill Package Structure 12
4.4 Plugin Package Structure 13
4.5 The Self-Bootstrapping Property 14
5 Signing Protocol 14
5.1 Signed Artifact Format 14
5.1.1 Skills 14
5.1.2 Plugins 15
5.2 Manifest Algorithm 15
5.3 The Convergence Loop 16
5.4 Step-by-Step Signing Flow 17
6 Verification Protocol 18
6.1 Key Discovery 18
6.2 Temporary Keyring 18

6.3 Step-by-Step Verification Flow . . . 19
6.3.1 Plugin Verification Differences . . . 19
7 Trust Model . . . 20
7.1 Path 1: Trusted Author . . . 21
7.2 Path 2: Trusted Attester, Signed Skill . . . 21
7.3 Path 3: Trusted Attester, Unsigned Skill . . . 21
7.4 Path 4: Destatement (Blocking) . . . 21
7.4.1 Per-Skill Overrides . . . 22
7.4.2 Trust Bundles . . . 22
7.5 Trust Store . . . 22
7.6 Policy Engine . . . 23
7.6.1 Actions . . . 24
7.6.2 Scenarios and Default Policies . . . 24
7.6.3 Decision Flow . . . 24
8 Attestation System . . . 25
8.1 Design Principles . . . 25
8.2 Bundle Format . . . 26
8.3 Canonical JSON . . . 27
8.4 Attestation Scopes . . . 27
8.5 Discovery Mechanisms . . . 27
8.5.1 Explicit Path or URL . . . 28
8.5.2 Local ATTESTATIONS/ Directory . . . 28
8.5.3 Reviewer Repository Convention . . . 28
8.6 Staleness . . . 28
9 Enforcement . . . 29
9.1 PreToolUse Hook Architecture . . . 29
9.2 Fail-Closed Model . . . 30
9.3 Plugin Resolution for Namespaced Skills . . . 30
10 Trust Store Hardening . . . 31
10.1 The Poisoning Threat . . . 31
10.2 Layer 1: GPG Passphrase Gating . . . 31
10.3 Layer 2: File Immutability . . . 31
10.4 Layer 3: Root Ownership . . . 32
10.5 Layer 4: Hook-Based Command Blocking . . . 32
10.6 Recommended Configuration . . . 32

10.7 Integrity Verification 32
11 Comparison with Existing Solutions 33
11.1 Landscape Matrix 33
11.2 Why Not Keyless Signing? 33
11.3 The Gap SkillSeal Fills 34
12 Adoption Perspectives 34
12.1 Skill Authors 35
12.2 Security Reviewers 35
12.3 Open Source Communities 36
12.4 Enterprise Environments 36
13 Security Analysis 37
13.1 Timing-Safe Comparisons 37
13.2 Input Validation 37
13.3 Process Isolation 38
13.4 Known Limitations 38
13.5 Key Cache 38
13.6 GPG Revocation Detection 39
13.7 Attestation Liveness Probe 39
14 Future Work 39
15 Conclusion 40
16 Appendix A: Specifications 40
17 Appendix B: CLI Reference 41

1 Abstract

Large language model agents execute skills and plugins — Markdown documents, command definitions, hook scripts, and agent configurations that function as installers with full system privileges. No standard mechanism exists to verify who authored these artifacts or whether they have been modified after publication. Independent audits of major agent skill repositories have found 12–20% of published skills to be actively malicious.

SkillSeal addresses this gap with a lightweight, self-bootstrapping cryptographic signing framework for LLM agent skill packages and plugins. Authors sign artifacts with multiple keys simultaneously — GPG and SSH — using a pluggable provider architecture that allows new signing methods without modifying core code. Keys are discoverable through GitHub's existing public key infrastructure (GPG keys via `github.com/{user}.gpg`, SSH signing keys via the GitHub API). A SHA-256 manifest ensures file-level integrity. A local trust store with configurable policy enables agents to make deterministic trust decisions without user intervention for known authors and reviewers. Independent reviewers can add attestations — multi-key-signed statements pinned to exact artifact digests — building a decentralized web of trust.

A PreToolUse hook for Claude Code enforces verification at the point of execution with a fail-closed security model: any skill that fails verification is blocked before it can run. Trusted reviewers can publish destatements — negative attestations that block execution regardless of author trust — and users can subscribe to community-curated trust bundles for scalable trust distribution. Verification requires only one valid signature to pass, so verifiers need only one of the author's key types.

SkillSeal operates at the artifact layer, complementing rather than replacing transport-layer solutions (OAuth, TLS), gateway products, and container isolation. It fills a specific gap in the current landscape: no existing tool provides a portable, self-bootstrapping, artifact-level signing standard that individual authors can adopt today.

2 Introduction

2.1 The Unsigned Executable Content Problem

LLM agents are capable instruction followers. When given a skill — a set of instructions describing how to accomplish a task — an agent will execute those instructions to achieve the user's stated goal. The agent optimizes for task completion, not security. It does not independently evaluate whether the *path* to completing a task is safe.

A skill is executable content. If a skill contains installation instructions, the agent follows them. If a skill tells the agent to fetch a binary from a URL and add it to PATH, the agent complies — it is achieving the user's intent. The side effect may be that malware is installed alongside a working calendar widget.

This is the confused deputy problem with an LLM as the deputy.

The attack pattern is straightforward:

1. Publish a skill that promises useful functionality
2. Include instructions that also perform malicious actions
3. The agent executes both — the useful part satisfies the user, the malicious part operates silently

2.2 The Scale of the Problem

The threat is not theoretical. Independent audits and security research have documented widespread exploitation:

OpenClaw skill repository compromise. Audits of OpenClaw's ClawHub — the largest open-source agent skill repository with 190,000+ GitHub stars — found 386 of 2,857 skills (12–20%) were actively malicious, primarily delivering Atomic Stealer (AMOS) malware to macOS. A skill called "What Would Elon Do?" was gamed to the number-one position while silently exfiltrating data. Over 135,000 internet-facing OpenClaw instances were discovered leaking plaintext API keys and chat histories. Three CVEs were assigned, including one-click remote code execution (CVSS 8.8).

Slopsquatting. Analysis of 756,000 AI-generated code samples found that approximately 20% recommend packages that do not exist. Attackers register these hallucinated package names on npm and PyPI with malware payloads, turning AI coding assistants into malware distribution vectors.

MCP tool poisoning. Research demonstrates that malicious instructions embedded in MCP tool descriptions achieve 72–84% attack success rates against frontier models. More capable models are more vulnerable — Claude 3.7 Sonnet's refusal rate was under 3%. SkillSeal does not address this attack vector — it operates at the artifact layer, not the protocol layer. However, knowing *who published* an MCP server configuration is a prerequisite for accountability.

Rules file backdoor. Invisible Unicode characters in AI coding assistant configuration files cause agents to silently produce backdoored code. The attack persists across sessions and survives project forking.

s1ngularity/Nx compromise. The first documented malware weaponizing AI coding agents — invoking Claude Code, Gemini CLI, and Amazon Q with unsafe flags and embedded exfiltration prompts.

As 1Password's research team observed: "Markdown isn't 'content' in an agent ecosystem. Markdown is an installer."

2.3 Core Thesis

Signing does not prove code is safe. It proves **provenance** and **integrity**. That distinction is critical.

An agent's trust decision requires answers to four questions:

- **Who authored this skill?** (provenance)
- **Has it been tampered with since authoring?** (integrity)
- **Do I have reason to trust that author?** (trust chain)
- **Has anyone I trust reviewed this?** (attestation)
- **Has anyone I trust flagged this as dangerous?** (destatement)

Cryptographic signing provides the first two and the mechanism for the second two. GitHub stars provide a weak, gameable popularity signal. A cryptographic signature provides an identity that can accumulate reputation over time and be verified mechanically.

2.4 Scope

SkillSeal operates at the **artifact layer**. It signs and verifies the skill packages and plugins that agents execute. It does not address:

- Transport-layer security (OAuth, TLS) — these are complementary
- Container isolation — SkillSeal identifies *what* is running, not *how* to sandbox it
- Protocol-level MCP security — SkillSeal does not inspect, validate, or secure MCP tool descriptions or server behavior. If a plugin includes an `.mcp.json` configuration, it is covered by the manifest hash (tamper detection), but SkillSeal has no understanding of what the MCP tools do
- Centralized registries — SkillSeal is decentralized by design

3 Threat Model

SkillSeal defends against six primary attack vectors. Each maps to a specific defense mechanism.

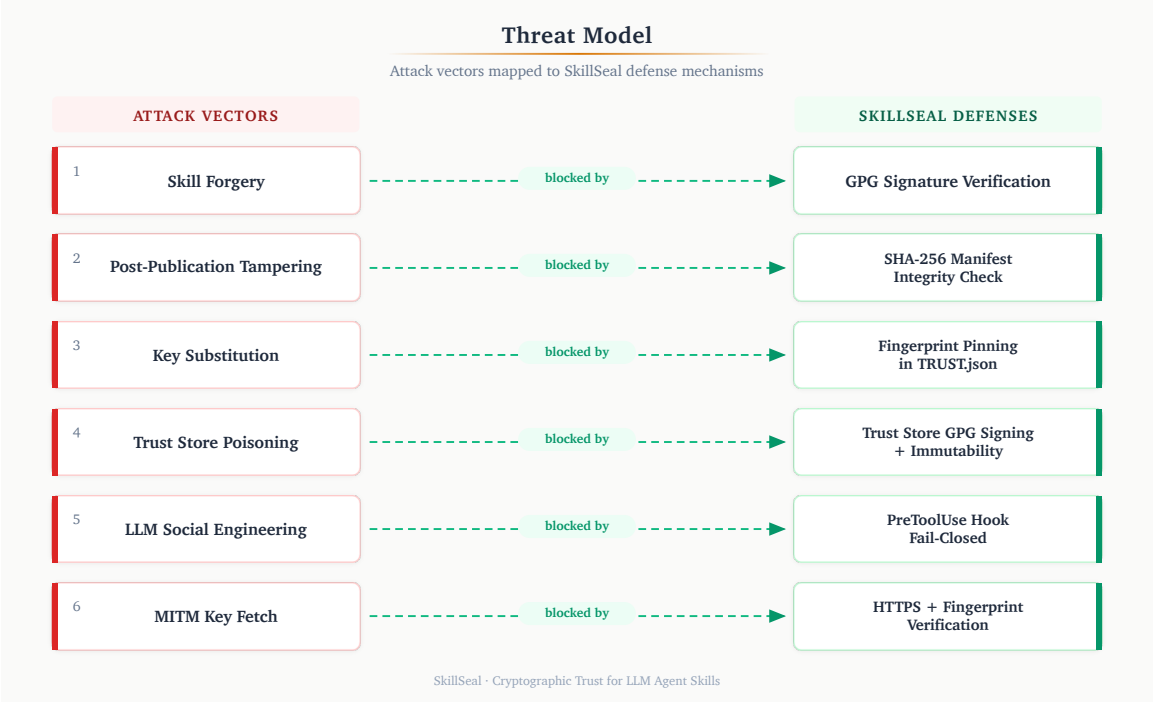

Figure 1: SkillSeal threat model: six attack vectors mapped to six defense mechanisms.

3.1 Attack Vectors and Defenses

3.1.1 Skill Forgery

Attack: An adversary publishes a malicious skill under a false identity, impersonating a trusted author.

Defense: Cryptographic signature verification. The skill's detached signatures (in the `SIGNATURES/` directory — `gpg.sig`, `ssh.sig`, etc.) are verified against the author's public keys fetched from GitHub. A forged skill cannot produce a valid signature without the author's private key. Only one valid signature is required to pass verification.

3.1.2 Post-Publication Tampering

Attack: An adversary modifies files in a skill package after the author has signed it — injecting malicious instructions, altering referenced scripts, or adding exfiltration code.

Defense: SHA-256 manifest integrity. The `MANIFEST.json` file records the SHA-256 digest of every file in the package. The manifest's own digest is embedded in the signed artifact's frontmatter. Any file modification, addition, or deletion is detected.

3.1.3 Key Substitution

Attack: An adversary intercepts the key discovery process and substitutes their own public key for the author's, allowing their forged signature to verify.

Defense: Fingerprint pinning. The `TRUST.json` file in each package records the expected GPG fingerprint. After fetching the key from GitHub, SkillSeal verifies that the imported key's fingerprint matches the pinned value. A substituted key with a different fingerprint is rejected.

3.1.4 Trust Store Poisoning

Attack: A compromised agent or malicious skill instructs the LLM to run `skillseal trust add` with an attacker's fingerprint, adding the attacker as a trusted author. Subsequent malicious skills signed by the attacker pass verification.

Defense: Four-layer trust store hardening (detailed in Section 9): GPG-signed trust store requiring passphrase for modification, file immutability flags, root ownership, and PreToolUse hook command blocking.

3.1.5 LLM Social Engineering

Attack: Prompt injection in a skill's instructions attempts to convince the agent to skip verification, ignore hook results, or treat an unsigned skill as trusted.

Defense: PreToolUse hook enforcement (detailed in Section 8). The hook executes as a separate process outside the LLM's context. The LLM cannot influence the hook's decision — it runs `skillseal verify` independently and returns `exit 0` (allow) or `exit 2` (block). The hook is fail-closed: any error blocks execution.

3.1.6 Man-in-the-Middle Key Fetch

Attack: An adversary with network position (corporate proxy, compromised CDN edge, DNS poisoning) intercepts the HTTPS request to `github.com/{username}.gpg` and serves a different key.

Defense: HTTPS transport provides baseline protection. Fingerprint pinning in `TRUST.json` provides defense-in-depth — even if the key fetch is intercepted, the substituted key's fingerprint will not match the expected value. The combination means an attacker must compromise both the transport layer and the pinned fingerprint.

3.2 What SkillSeal Does Not Prevent

- **A trusted author publishing malicious code.** Signing proves identity, not intent. A trusted author who publishes a malicious update will pass verification. The current policy engine does not require attestations for trusted authors — `known_author_no_attestations` defaults to `allow`. The mitigation is to revoke trust (`skillseal trust remove`) when an author is compromised or acts maliciously. Additionally, trusted reviewers can publish destatements to flag dangerous skills for all users who trust them — providing a community-level response mechanism.
- **Key compromise.** If an author's private key is stolen, the attacker can produce valid signatures. GPG key revocation is detected as of v0.2.5. SSH key revocation is planned.
- **Vulnerabilities in skill logic.** SkillSeal verifies provenance and integrity, not functional correctness. A skill that accidentally exposes an SSRF vector will pass verification if the author signed it.
- **Zero-day exploitation of GPG itself.** SkillSeal delegates cryptographic operations to GPG. A vulnerability in GPG affects all systems that depend on it.

4 Architecture

4.1 Design Principles

SkillSeal's architecture is guided by five principles:

Lightweight. The entire system is a CLI tool, a trust store, and a hook script. No daemons, no servers, no databases. Signing and verification complete in milliseconds.

Self-bootstrapping. The verification skill (`skillseal-verify/SKILL.md`) is itself signed. An agent that trusts this one artifact gains the capability to verify everything else — analogous to a root CA certificate shipped with an operating system.

Decentralized. No central authority controls who can sign or attest. Authors sign with their own keys (GPG, SSH, or both). Reviewers publish their own attestations. Trust decisions are local to each user's trust store.

Multi-key. Authors sign with multiple key types simultaneously via a pluggable provider architecture. Each signing operation produces one signature per configured key in a `SIGNATURES/` directory. Verification requires only one valid signature, so verifiers need only one of the author's key types. New providers can be added without modifying core code.

Composable. SkillSeal operates at the artifact layer and composes with any transport, gateway, or isolation mechanism. An enterprise proxy can consume SkillSeal signatures alongside its own policy. A container runtime can verify signatures before mounting a skill volume.

Fail-closed. Every error condition — missing files, parse failures, network errors, GPG crashes — results in blocking the skill, not allowing it. The system defaults to denying execution.

4.2 Components

Component	Purpose
`skillseal sign <dir>`	Sign a skill or plugin (auto-detected)
`skillseal verify <dir>`	Verify signature, manifest, and trust policy
`skillseal sign-all <dir>`	Batch-sign all skills and plugins in a directory
`skillseal attest <dir>`	Create a reviewer attestation bundle
`skillseal init <dir>`	Scaffold a new skill package
`skillseal trust <cmd>`	Manage trust store, overrides, and bundles
`skillseal cache-clear`	Clear cached credentials for all providers
`hooks/skill-verify.ts`	PreToolUse enforcement hook for Claude Code
`skillseal-verify/SKILL.md`	Skill teaching LLMs to verify (self-signed)
`skillseal-sign/SKILL.md`	Skill teaching LLMs to sign (self-signed)

4.3 Skill Package Structure

A signed skill package contains four files alongside the skill content:

```
my-skill/
├── SKILL.md          # The skill instructions (signed artifact)
├── SIGNATURES/       # One signature per key type
│   ├── gpg.sig       # Detached GPG signature
│   └── ssh.sig       # SSH signature (ssh-keygen -Y sign)
├── MANIFEST.json     # SHA-256 hashes of all package files
├── TRUST.json        # Author identity with keys[] array
└── ATTESTATIONS/     # Reviewer and scanner attestation bundles
```

`SKILL.md` is the signed artifact — the single file whose cryptographic signatures anchor the entire package. The manifest hash embedded in its YAML frontmatter binds all other files to the signatures. Each key type produces its own signature in the `SIGNATURES/` directory.

4.4 Plugin Package Structure

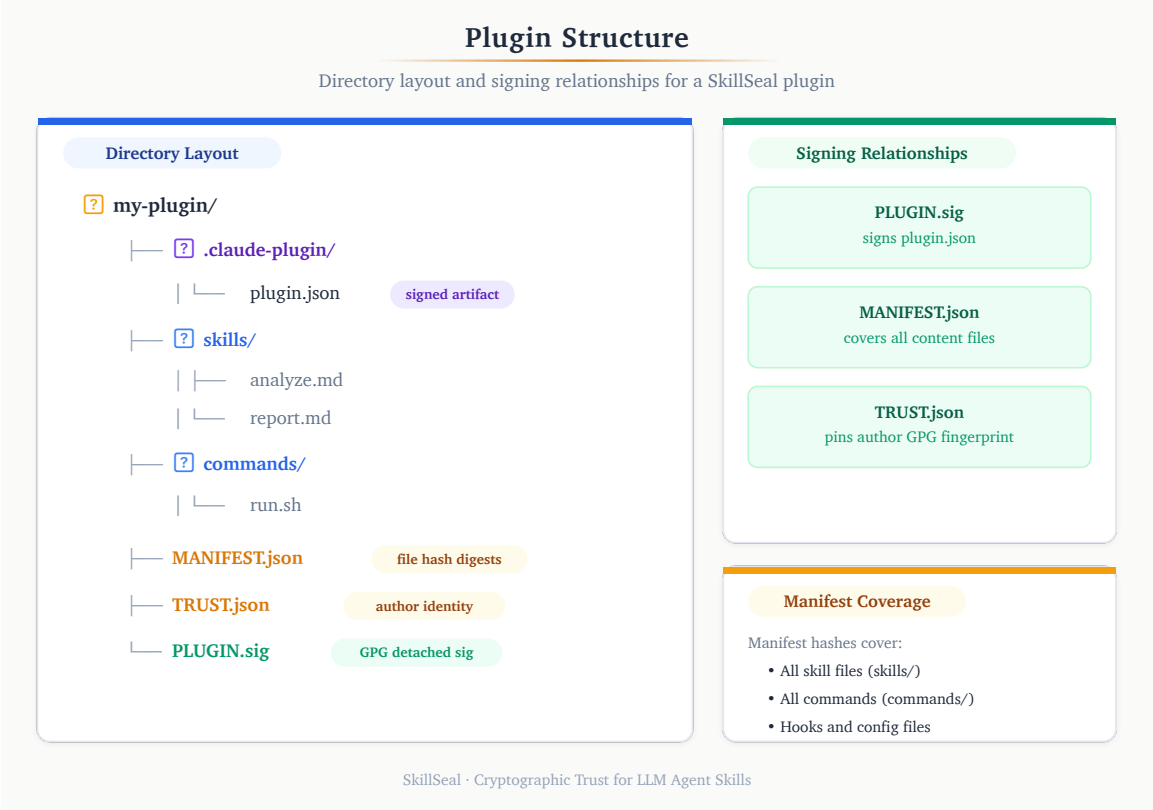

Figure 2: Plugin directory structure with signing relationships.

Plugins are the distribution unit for Claude Code extensions. A plugin bundles skills, commands, hooks, and agents into a single package. If an `.mcp.json` configuration is present, it is included in the manifest like any other file — SkillSeal verifies its integrity but does not inspect its contents. SkillSeal signs the entire plugin as one unit:

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json   # Plugin metadata (signed artifact)
├── skills/           # Skill packages
├── commands/         # Slash command definitions
├── hooks/            # Hook scripts
├── agents/           # Agent definitions
├── SIGNATURES/       # One signature per key type
│   ├── gpg.sig       # Detached GPG signature of plugin.json
│   └── ssh.sig       # SSH signature of plugin.json
├── MANIFEST.json     # SHA-256 hashes of all plugin files
├── TRUST.json        # Author identity with keys[] array
└── README.md
```

The `sign` and `verify` commands auto-detect whether a directory is a plugin (contains `.claude-plugin/plugin.json`) or a standalone skill (contains `SKILL.md`). One signature covers all contents.

4.5 The Self-Bootstrapping Property

A critical architectural property: the system can verify itself before any external trust is established.

The SkillSeal verification skill (`skillseal-verify/SKILL.md`) is signed with standard GPG. A user can verify it using raw GPG commands — no SkillSeal CLI required:

```
curl -sL https://github.com/mcyork.gpg | gpg --import
gpg --verify skillseal-verify/SIGNATURES/gpg.sig skillseal-verify/SKILL.md
```

Once this single manual trust decision is made, the agent has the capability to verify every subsequent skill autonomously. This is the same pattern as a root CA certificate: one trust anchor bootstraps the entire system.

5 Signing Protocol

5.1 Signed Artifact Format

5.1.1 Skills

The signed artifact is `SKILL.md`. Its YAML frontmatter contains the fields that bind the signature to the package:

```
---
skill: ssl-certificate-checker
version: 1.0.0
author: ian@esoup.net
github: mcyork
signed: true
manifest_hash: sha256:a1b2c3d4e5f6...
---
```

Author identity and key metadata are stored in `TRUST.json` rather than the frontmatter, supporting multiple keys:

```
{
  "schema_version": "0.2.5",
  "author": {
    "name": "Ian McCutcheon",
    "github": "mcyork",
    "keys": [
      { "type": "gpg", "fingerprint": "7097CE1E...", "key_url": "https://
github.com/mcyork.gpg" },
      { "type": "ssh", "fingerprint": "SHA256:vZci...", "key_url": "https://
api.github.com/users/mcyork/ssh_signing_keys" }
    ]
  }
}
```

5.1.2 Plugins

The signed artifact is `.claude-plugin/plugin.json`. Signing metadata is embedded directly in the JSON:

```
{
  "name": "skillseal-demo-plugin",
  "version": "1.0.0",
  "description": "SSL certificate checker plugin",
  "author": { "name": "Ian McCutcheon", "email": "ian@esoup.net" },
  "signed": true,
  "manifest_hash": "sha256:504861f3a997..."
}
```

5.2 Manifest Algorithm

The manifest records SHA-256 digests of all files in the package directory, with specific exclusions:

Excluded from skill manifests:

- `SIGNATURES/` directory (the signatures themselves)
- `MANIFEST.json` (the manifest itself — would create circularity)
- `TRUST.json` (written independently; integrity bound through the signature chain)
- `ATTESTATIONS/` directory (independently verifiable)
- Hidden files and directories (names starting with `.`)

Excluded from plugin manifests:

- `SIGNATURES/`, `MANIFEST.json`, `TRUST.json` (same rationale)
- `.claude-plugin/plugin.json` (the signed artifact — excluded to break circularity)
- `ATTESTATIONS/`, `.git/`, `node_modules/`

The manifest format:

```
{
  "schema_version": "0.2.5",
  "generated_at": "2026-02-14T00:00:00Z",
  "algorithm": "sha256",
  "files": {
    "SKILL.md": "abcdef1234567890...",
    "src/helper.py": "fedcba0987654321..."
  }
}
```

File paths use forward slashes and are relative to the package root.

5.3 The Convergence Loop

A circular dependency exists between the signed artifact and the manifest: the manifest hash is embedded in `SKILL.md` (or `plugin.json`), but `SKILL.md` is itself a file whose content affects the manifest.

SkillSeal resolves this with an iterative convergence loop:

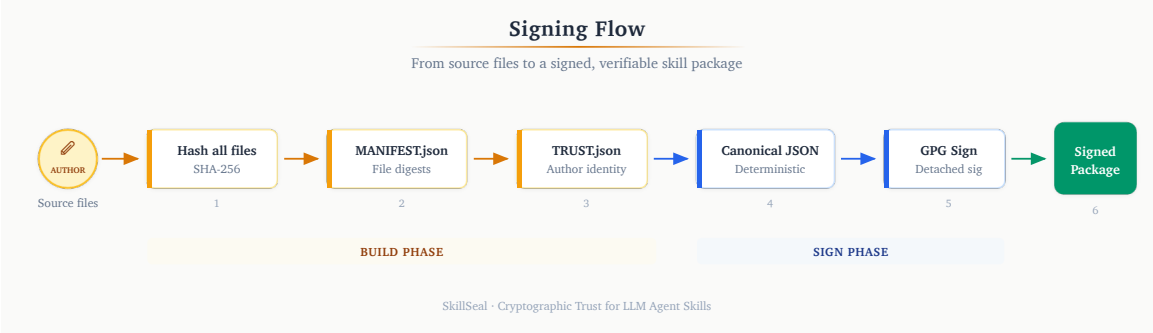

Figure 3: Signing flow with convergence loop.

1. Write `TRUST.json` with author identity and fingerprint
2. Walk the directory and compute SHA-256 hashes of all eligible files
3. Write `MANIFEST.json` with the computed hashes
4. Compute SHA-256 of `MANIFEST.json` and embed it in the signed artifact's frontmatter
5. Check if the manifest hash has changed since the last iteration
6. If changed, return to step 2 (the signed artifact's content changed, which changes its hash in the manifest)

7. If stable, sign the artifact with GPG

For skills, convergence typically occurs in 2–3 iterations. For plugins, convergence is immediate (one iteration) because `plugin.json` is excluded from the manifest — its content changes do not affect the manifest hash.

5.4 Step-by-Step Signing Flow

1. **Auto-detect package type.** Check for `.claude-plugin/plugin.json` (plugin) or `SKILL.md` (skill).
2. **Write TRUST.json.** Record author name, email, GitHub username, fingerprint, and key URL.
3. **Enter convergence loop:**
   a. Walk directory, compute SHA-256 hashes (respecting exclusion sets)
   b. Write `MANIFEST.json`
   c. Compute `sha256:` prefixed digest of `MANIFEST.json`
   d. Update the signed artifact's frontmatter with the manifest hash
   e. If manifest hash differs from previous iteration, repeat
4. **Sign with all configured keys.** For each key in the author's config, produce a signature in the `SIGNATURES/` directory:
   - GPG: `gpg --detach-sign --armor --output SIGNATURES/gpg.sig SKILL.md`
   - SSH: `ssh-keygen -Y sign -f <key_path> -n skillseal SKILL.md` → `SIGNATURES/ssh.sig`
   - Plugins follow the same pattern against `.claude-plugin/plugin.json`

Supported GPG key types: RSA (2048+ bit), Ed25519, Ed448. Supported SSH key types: Ed25519 (recommended, always accepted), RSA (3072+ bit, minimum 128-bit security). DSA keys are rejected.

6 Verification Protocol

6.1 Key Discovery

Author public keys are discovered through GitHub's existing key endpoints:

- **GPG keys:** `https://github.com/{username}.gpg`
- **SSH signing keys:** `https://api.github.com/users/{username}/ssh_signing_keys` (filtered by keys with title exactly matching "skillseal", case-insensitive)

This requires no custom infrastructure. Developers who sign git commits already have keys published here. For SSH, SkillSeal validates key strength (minimum 128-bit security) and filters for keys explicitly designated for SkillSeal use.

The design deliberately avoids custom key servers, `.well-known` paths, or centralized registries. GitHub is where developers publish code and where skills are hosted — using it as the key directory eliminates a class of deployment friction.

6.2 Temporary Keyring

Verification uses an ephemeral GPG keyring to avoid polluting the user's default keyring:

1. Create a temporary directory via `mkdtemp` with permissions `0700`
2. Import the fetched key material into this temporary keyring (`--homedir`)
3. Perform all verification operations against the temporary keyring
4. Delete the temporary directory in a `finally` block (cleanup on both success and error)

This ensures that verifying a skill never has side effects on the user's GPG state.

6.3 Step-by-Step Verification Flow

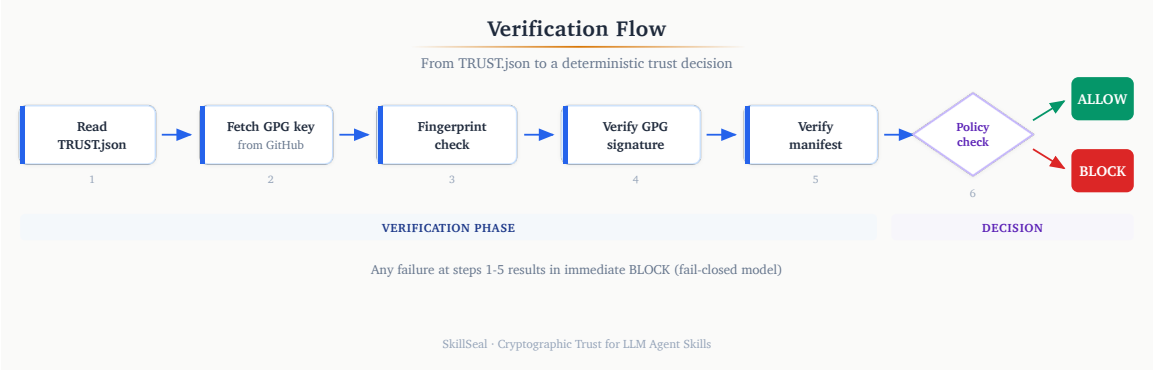

Figure 4: Verification flow from TRUST.json to policy decision.

1. **Read TRUST.json.** Extract `author.github` (GitHub username) and `author.fingerprint` (expected GPG fingerprint).
2. **Fetch public key.** Download `https://github.com/{author.github}.gpg`. Validate that the response contains a PGP public key block.
3. **Import into temporary keyring.** Import the fetched key material into the ephemeral GPG home directory.
4. **Verify fingerprint.** Locate the imported key matching `author.fingerprint`. If no match, reject — the fetched key does not correspond to the claimed identity.
5. **Verify signatures.** Read the `SIGNATURES/` directory and attempt verification with each available provider. For GPG: `gpg --verify SIGNATURES/gpg.sig SKILL.md`. For SSH: `ssh-keygen -Y verify` against `SIGNATURES/ssh.sig`. Only one valid signature is required to pass.
6. **Verify manifest integrity:**
   a. Read `MANIFEST.json` and recompute SHA-256 of every listed file
   b. Compare computed hashes against stored hashes
   c. Check for unlisted files (files present in the directory but not in the manifest, excluding exempt paths)
7. **Evaluate trust policy.** Look up the author in the trust store. Check for attestations from trusted reviewers. Apply the appropriate policy action (allow, prompt, refuse).
8. **Clean up.** Delete the temporary keyring.

6.3.1 Plugin Verification Differences

Plugin verification follows the same flow with two substitutions:

- The signed artifact is `.claude-plugin/plugin.json` instead of `SKILL.md`

- The `SIGNATURES/` directory contains signatures of `plugin.json` rather than `SKILL.md`
- The manifest uses the plugin exclusion set (allowing traversal into dot-prefixed directories like `.claude-plugin/`)

Auto-detection means the user runs the same command for both: `skillseal verify <dir>`.

7 Trust Model

The trust model defines three paths to allowing a skill to execute and one path that blocks. Any positive path is sufficient for access; a destatement overrides all positive paths.

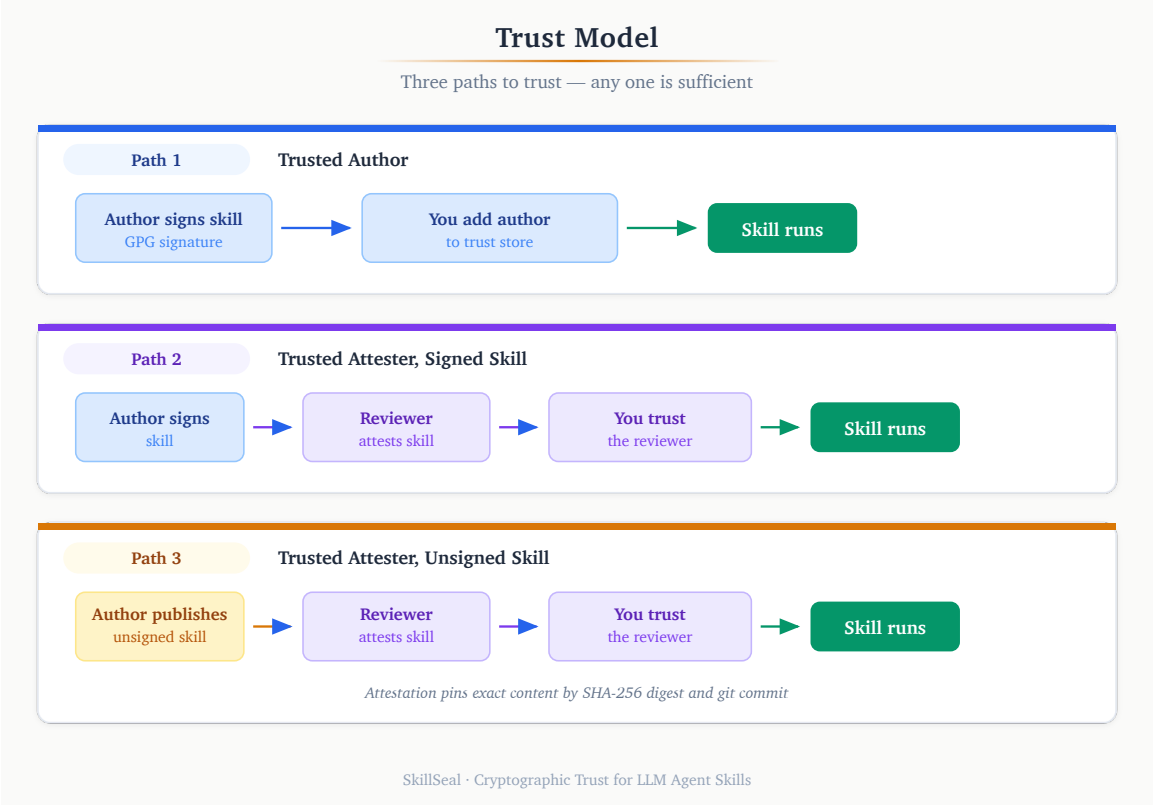

Figure 5: Three paths to trust: trusted author, trusted attester with signed skill, trusted attester with unsigned skill.

7.1 Path 1: Trusted Author

The skill is signed by an author listed in the user's trust store. The signature proves the author wrote this specific version and that it has not been tampered with.

```
Author signs skill → Author is in trust store → ALLOW
```

This is the simplest path. If you trust the author, and the signature is valid, the skill runs.

7.2 Path 2: Trusted Attester, Signed Skill

The skill is signed by an unknown author, but a reviewer you trust has independently examined the skill and published an attestation. The attestation is a GPG-signed statement pinned to the exact SHA-256 digests of the skill's files.

```
Author signs skill → Trusted reviewer attests → ALLOW
```

The user never needs to trust the author directly. The reviewer's attestation is the trust anchor. This enables a division of labor: security teams or trusted individuals review skills, and everyone else benefits from their review.

7.3 Path 3: Trusted Attester, Unsigned Skill

The author did not sign the skill at all — no `SIGNATURES/` directory, no `TRUST.json`, no `MANIFEST.json`. A trusted reviewer attested it anyway: they reviewed the content, pinned it by SHA-256 digest and git commit, and signed a statement.

```
Author publishes skill (unsigned) → Trusted reviewer attests → ALLOW
```

This path enables verification of third-party skills from authors who have not adopted SkillSeal. The attestation pins the exact content that was reviewed. Any modification after attestation makes the attestation stale.

7.4 Path 4: Destatement (Blocking)

A trusted reviewer has examined a skill and published a **destatement** — an attestation bundle with `verdict: "reject"`. This signals that the reviewer has identified a problem with the skill.

```
Reviewer publishes destatement → Reviewer is in trust store → BLOCK
```

Destatements are checked **before** any positive trust evaluation. A skill that is signed by a trusted author, attested by three trusted reviewers, and destatement'd by one trusted reviewer is blocked. The destatement overrides all positive signals.

This gives reviewers the power to flag dangerous skills. When a vulnerability is discovered in a published skill, a reviewer can publish a destatement that immediately blocks execution for all users who trust that reviewer — without requiring the author to take any action.

7.4.1 Per-Skill Overrides

If a user disagrees with a specific destatement, they can add a per-skill override to their trust store:

```
skillseal trust override add my-skill --despite reviewer-github --reason "We
reviewed and disagree"
```

The override applies only to the specific combination of skill name and reviewer. The destatement still exists and is still visible — it is simply bypassed for that skill in that user's trust store.

7.4.2 Trust Bundles

Community curators and organizations can publish **trust bundles** — signed JSON files containing curated lists of trusted authors and reviewers, hosted on GitHub repositories:

```
skillseal trust bundle add org/trust-bundle-repo
skillseal trust bundle update
```

On update, SkillSeal fetches the bundle, verifies the publisher's signature against the local trust store, and merges new entries without overwriting existing ones. Revoked fingerprints in the bundle remove matching keys from the local store. This enables scalable trust distribution — a security team can maintain a bundle that all members subscribe to.

7.5 Trust Store

The trust store is a local JSON file at `~/.skillseal/trust-store.json` that records trusted authors, trusted reviewers, and policy configuration:

```
{
  "schema_version": "0.2.5",
  "trusted_authors": {
    "mcyork": {
      "keys": [
        { "type": "gpg", "fingerprint":
"7097CE1EF54E0808FD3855427ED9682FF64286D0" },
        { "type": "ssh", "fingerprint": "SHA256:vZcivMOtxMdRjvcyGpNSjECXhb/
wspMSsHO/bfPXBmQ" }
      ],
      "trust_level": "author",
      "added_at": "2026-02-14T00:00:00Z"
    }
  },
  "trusted_reviewers": {
    "security-team": {
      "keys": [
        { "type": "gpg", "fingerprint": "FEDCBA0987654321..." }
      ],
      "trust_level": "reviewer"
    }
  },
  "policies": { ... },
  "overrides": [],
  "bundles": []
}
```

Trusted entities support multiple keys via the `keys[]` array. Verification checks if ANY key in the entity matches the signature's fingerprint, enabling authors and reviewers to use different key types over time without losing trust relationships.

The trust store is conceptually identical to a browser's CA certificate store. It is the root of all trust decisions — its integrity is critical and protected by multiple hardening layers (Section 9).

7.6 Policy Engine

The policy engine maps seven trust scenarios to four possible actions:

7.6.1 Actions

Action	Behavior
`refuse`	Block the skill. Inform the user.
`prompt`	Ask the user for permission. In a PreToolUse hook context, this is treated as `refuse` (hooks cannot prompt interactively).
`allow`	Run the skill.
`install_silently`	Run the skill without notice.

7.6.2 Scenarios and Default Policies

Scenario	Condition	Default
`unsigned`	No signature, no TRUST.json, no valid attestation	`refuse`
`signature_invalid`	Signature present but fails verification	`refuse`
`unknown_author`	Valid signature, author not in trust store, no trusted attestation	`prompt`
`known_author_no_attestations`	Trusted author, no attestations	`allow`
`known_author_with_attestations`	Trusted author, attestations present but reviewers not trusted	`allow`
`known_author_stale_attestations`	Trusted reviewer attested, but attestation is for an older version	`prompt`
`trusted_reviewer_attested`	Trusted reviewer has a current, valid attestation	`allow`
`trusted_reviewer_destatement`	A trusted reviewer published a destatement (verdict: “reject”)	`refuse`

7.6.3 Decision Flow

The key principle: **attestation trust overrides author trust.** If a trusted reviewer has attested a skill, it is allowed regardless of whether the author is known.

```
Skill discovered
  │
  ├── Signature invalid? → "signature_invalid"
  │
  ├── *** CHECK DESTATEMENTS FIRST ***
  │   Any trusted reviewer destatement (verdict: "reject")?
  │     ├── Yes → Override exists for this skill + reviewer?
  │     │     ├── Yes → Skip this destatement, continue
  │     │     └── No  → "trusted_reviewer_destatement" (BLOCKS)
  │     └── No → Continue to positive trust evaluation
  │
  ├── Has signature and TRUST.json?
  │     ├── No → Has valid attestation from trusted reviewer?
  │     │         ├── Yes (current) → "trusted_reviewer_attested"
  │     │         ├── Yes (stale)   → "known_author_stale_attestations"
  │     │         └── No            → "unsigned"
  │     └── Yes → Check for trusted reviewer attestation
  │           ├── Current → "trusted_reviewer_attested"
  │           ├── Stale   → "known_author_stale_attestations"
  │           └── None    → Check author in trust store
  │                 ├── Unknown → "unknown_author"
  │                 └── Known → Check attestation presence
  │                       ├── None     → "known_author_no_attestations"
  │                       └── Untrusted → "known_author_with_attestations"
  │
```

This produces the same UX gradient as macOS's application signing: "identified developer" flows through, "unidentified developer" requires explicit permission, and unsigned applications are blocked by default.

8 Attestation System

8.1 Design Principles

The attestation system is built on four principles:

Decoupled. The reviewer creates and hosts attestations independently. The skill author is never involved in the attestation process and cannot prevent or control it.

Content-addressed. Attestations pin the exact bytes of the signed artifact and manifest via SHA-256 digests. They are not tied to mutable references like branch names, tags, or repository URLs.

Self-contained. A single `.attestation.json` file contains the statement, reviewer identity, and GPG signature. No external dependencies are required to verify it.

Staleness-aware. When a skill's files change, existing attestations become stale. This is correct behavior — analogous to a pull request approval being invalidated by new commits. The staleness is detected and reported; the trust store policy determines the response.

8.2 Bundle Format

An attestation bundle is a JSON file with the extension `.attestation.json`:

```
{
  "schema_version": "0.2.5",
  "format": "skillseal-attestation-bundle/v1",
  "statement": {
    "type": "https://skillseal.dev/attestation/review/v1",
    "subject": {
      "skill": "skillseal-demo-plugin",
      "version": "1.0.0",
      "repository": "github.com/mcyork/skillseal-demo-plugin",
      "commit": "61f027aa9ca03086e5c6b9d4...",
      "digests": {
        "skill_md_sha256": "bea2229b5ddaa2cc...",
        "manifest_sha256": "504861f3a997aa1c..."
      }
    },
    "created_at": "2026-02-15T01:13:34.774Z",
    "reviewer": {
      "name": "Ian McCutcheon",
      "github": "mcyork",
      "fingerprint": "7097CE1EF54E0808FD38..."
    },
    "attestation": {
      "scope": "full-review",
      "verdict": "approve",
      "statement": "Reviewed all plugin contents. Safe.",
      "date": "2026-02-15T01:13:34.774Z"
    }
  },
  "signatures": [
    { "type": "gpg", "value": "-----BEGIN PGP SIGNATURE-----\n..." },
    { "type": "ssh", "value": "-----BEGIN SSH SIGNATURE-----\n..." }
  ]
}
```

8.3 Canonical JSON

The signature in an attestation bundle signs the `statement` field, not the entire bundle. To ensure deterministic serialization (so the same statement always produces the same bytes for signature verification), SkillSeal uses canonical JSON:

- Object keys sorted lexicographically at every nesting level
- 2-space indentation
- Trailing newline at end of string
- No trailing commas
- Standard JSON encoding

This means a verifier can re-serialize the statement from the parsed bundle and verify the signature against the canonical bytes — even if the bundle was reformatted, whitespace was changed, or fields were reordered at the top level.

8.4 Attestation Scopes

Scope	Meaning
`full-review`	Reviewer examined all instructions and auxiliary files
`security-audit`	Focused security review (injection, exfiltration, prompt manipulation)
`automated-scan`	Machine-generated attestation from a scanning tool
`functional-review`	Verified that the skill works as described

Scopes are informational — the trust store does not currently distinguish between them for policy decisions. A future version may allow policies like "require `security-audit` attestation for skills that access the network."

8.5 Discovery Mechanisms

Attestations can be discovered through three mechanisms, in order of priority:

8.5.1 Explicit Path or URL

```
skillseal verify <dir> --attestation ./review.attestation.json
skillseal verify <dir> --attestation https://example.com/att.json
```

8.5.2 Local ATTESTATIONS/ Directory

Skills may include an `ATTESTATIONS/` subdirectory containing `.attestation.json` files. These are automatically discovered during `skillseal verify`. The `ATTESTATIONS/` directory is excluded from the manifest, so adding attestations does not invalidate the author's signature.

8.5.3 Reviewer Repository Convention

Reviewers host attestations in their own GitHub repository following a path convention:

```
github.com/{reviewer}/skillseal-attestations/
  {author}/{skill-name}/
    {version}.attestation.json
```

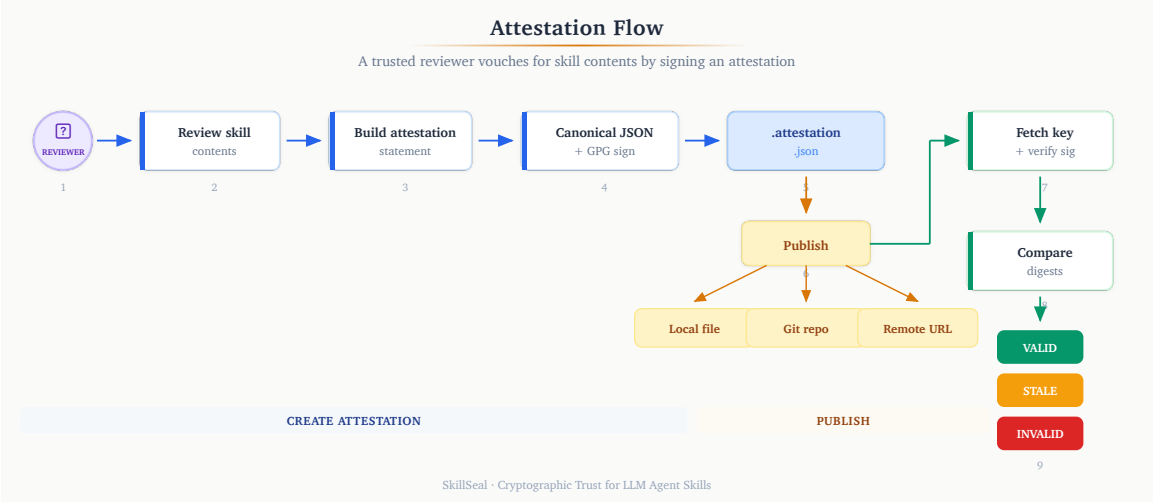

Figure 6: Attestation lifecycle: creation, publication, and verification.

8.6 Staleness

An attestation becomes stale when the skill's current SHA-256 digests no longer match the attested digests. This happens when the signed artifact or any manifested file is modified after attestation.

Stale attestations are still valid signatures — they refer to a different version of the skill. The verify output reports staleness:

```
mcyork (Ian McCutcheon): VALID [full-review] (v1.0.0 — STALE)
```

The trust store policy `known_author_stale_attestations` (default: `prompt`) controls behavior when all attestations from trusted reviewers are stale.

9 Enforcement

Signing and verification are only useful if enforced. SkillSeal ships a PreToolUse hook for Claude Code that blocks any skill from executing unless it passes verification.

9.1 PreToolUse Hook Architecture

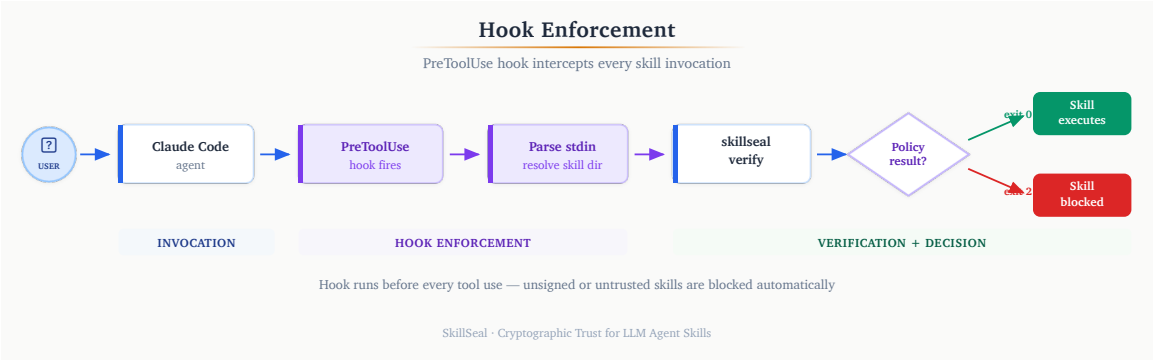

Figure 7: Hook enforcement flow: user invokes skill, hook intercepts, verifies, allows or blocks.

Claude Code's hook system allows external scripts to intercept tool invocations before execution. The SkillSeal hook:

1. Receives the tool invocation on stdin (JSON with `tool_name` and `tool_input`)
2. Filters for `Skill` tool invocations only
3. Resolves the skill name to a directory path (supporting both standalone skills and plugin-namespaced skills like `pluginName:skillName`)
4. Runs `skillseal verify` on the resolved directory
5. Parses the JSON output
6. Returns `exit 0` (allow) if the policy action is `allow` or `install_silently`

7. Returns `exit 2` (block) with an error message for any other result

When a skill is blocked, the hook outputs a facts-only message: the policy scenario that triggered the block, who flagged it (for destatements), the reason, and the date. No remediation commands or URLs are shown — the user decides how to respond.

9.2 Fail-Closed Model

The hook is fail-closed by design. Every error condition results in blocking:

- `skillseal verify` crashes or returns non-zero → block
- JSON output cannot be parsed → block
- Skill directory cannot be resolved → block
- Policy action is `prompt` → block (hooks cannot prompt interactively)
- Network error fetching keys → block
- Any unexpected condition → block

The agent cannot influence this decision. The hook runs as a separate process outside the LLM's context. Prompt injection in a skill's instructions cannot override the hook's verdict.

9.3 Plugin Resolution for Namespaced Skills

When a user invokes a skill from a plugin (e.g., `/pluginName:skillName`), the hook must locate the plugin directory. It scans the marketplace directories:

```
~/.claude/plugins/marketplaces/*/plugins/{pluginName}/
```

For each candidate directory, it checks for `.claude-plugin/plugin.json` to confirm it is a valid plugin. Verification then runs against the plugin root, covering all skills within it.

10 Trust Store Hardening

10.1 The Poisoning Threat

The trust store is the root of all trust decisions. If a compromised agent can modify it, the entire signing and verification system is bypassed:

1. Malicious skill instructs the LLM to run `skillseal trust add attacker-github ATTACKER_FINGERPRINT`
2. The trust store is re-signed automatically (signing key caches are warm)
3. Attacker signs a malicious skill → it now passes verification

The attack works because the LLM agent runs as the user and has the same file and GPG permissions.

10.2 Layer 1: GPG Passphrase Gating

Add `ignore-cache-for-signing` to `~/.gnupg/gpg-agent.conf`. This forces an interactive passphrase prompt on every trust store modification. LLM agents cannot enter passphrases — the operation fails with a TTY error.

This is the recommended minimum hardening. The passphrase prompt is the human-in-the-loop.

10.3 Layer 2: File Immutability

After configuring the trust store, lock the files using OS-level immutability:

macOS:

```
chflags uchg ~/.skillseal/trust-store.json ~/.skillseal/trust-store.json.sig
```

Linux:

```
sudo chattr +i ~/.skillseal/trust-store.json ~/.skillseal/trust-store.json.sig
```

The LLM agent cannot write to immutable files. To make legitimate changes, the user unlocks, modifies, and re-locks.

10.4 Layer 3: Root Ownership

Transfer ownership to root: `sudo chown root:wheel ~/.skillseal/trust-store.json*`. The agent process cannot modify root-owned files without `sudo`, which it cannot invoke.

10.5 Layer 4: Hook-Based Command Blocking

A PreToolUse hook on Bash commands can detect and block any command targeting the trust store:

```
const cmd = payload.tool_input?.command || "";
if (cmd.includes("trust-store.json") ||
    cmd.includes("skillseal trust add") ||
    cmd.includes("chflags nouchg")) {
  process.exit(2); // Block
}
```

This is defense-in-depth — even if other layers are misconfigured, the hook prevents the agent from running the commands.

10.6 Recommended Configuration

Environment	Layers
Personal, single user	Layer 1 + Layer 2
Shared machine or high-security	All four layers
Active development / experimenting	Layer 1 only

10.7 Integrity Verification

To verify the trust store has not been tampered with:

```
gpg --verify ~/.skillseal/trust-store.signatures/gpg.sig ~/.skillseal/trust-store.json
```

If GPG reports a bad signature, the trust store has been modified since it was last legitimately saved. SkillSeal itself detects this at load time and treats the store as empty (no trust, defaults only), printing a warning.

11 Comparison with Existing Solutions

11.1 Landscape Matrix

Solution	Layer	Signing	Prov.	Trust	Port.	Indiv.
SkillSeal	Artifact	GPG+SSH	Yes	Local	Yes	Yes
npm/Sigstore	Package	OIDC	Yes	Log	–	–
SLSA	Build	Build	Yes	Central	–	–
ETDI	Protocol	Crypto	Part.	Session	–	–
ToolHive	Gateway	Registry	Part.	Curated	–	–
Runlayer	Gateway	Platform	Part.	Platform	–	–
MCP Gateway	Isolation	–	–	Container	–	–
mcp-scan	Scanning	Hash	Part.	TOFU	Yes	Yes
Cloudflare	Transport	–	–	Zero-trust	–	–

Prov. = Provenance. Port. = Portable. Indiv. = Individual use. Part. = Partial. – = No.

11.2 Why Not Keyless Signing?

Sigstore's keyless signing model (used by npm, Python, and container registries) ties identity to OIDC tokens from identity providers. This eliminates key management but requires:

- A transparency log (Rekor) that must be online for verification
- An identity provider (GitHub, Google) to issue OIDC tokens at signing time
- A trusted root (Fulcio CA) to issue short-lived certificates

This is appropriate for package registries with centralized infrastructure. It is not appropriate for a decentralized, self-bootstrapping system where:

- Individual authors sign without any server infrastructure
- Verification can work offline (given cached keys)
- The trust model is local, not global

SkillSeal uses GPG and SSH because they enable decentralized, offline-capable signing with mature key distribution mechanisms (GitHub's existing endpoints). The pluggable provider architecture means new signing methods can be added without modifying core verification logic. The tradeoff is that authors must manage signing keys — but for LLM agents, key management is automatable in ways it never was for human email users. SSH keys in particular have near-zero adoption friction since developers already have them for git operations.

11.3 The Gap SkillSeal Fills

Existing solutions cluster into three categories:

Scanning-based tools (mcp-scan, VirusTotal, ClawSec) detect known-bad patterns after the fact. They cannot establish provenance or identity. A novel attack bypasses them.

Gateway and platform products (ToolHive, Runlayer, Docker MCP Gateway, Cloudflare) provide enforcement at the network boundary. They are not portable — they require specific infrastructure. An individual developer cannot use them.

Protocol-level proposals (ETDI, MACAW) extend MCP with per-session or per-call cryptographic identity. They do not address artifact-level trust — a signed MCP session can still serve a malicious tool definition.

SkillSeal occupies the gap between these: a **portable, self-bootstrapping, artifact-level signing standard** that works for individual authors today and composes with gateways and platforms tomorrow.

12 Adoption Perspectives

SkillSeal's five design principles — lightweight, self-bootstrapping, decentralized, composable, fail-closed — produce a system that serves different stakeholders without requiring separate features for each. The same primitives operate at every scale.

12.1 Skill Authors

A skill author's adoption path is minimal: configure signing keys (GPG, SSH, or both) and run `skillseal sign`. If the author already has an SSH key for git operations, they can start signing immediately with zero key generation overhead. The signed artifacts travel with the skill — no registry to publish to, no account to create, no CI pipeline to configure. The signatures are files alongside the code in a `SIGNATURES/` directory.

This matters because the adoption barrier determines whether signing actually happens. Every prior attempt at signing developer artifacts (PGP-signed emails, signed git tags) failed to achieve widespread adoption because the overhead exceeded the perceived benefit. SkillSeal's overhead is one command. The benefit is that agents can verify the author before executing the skill — which, given the threat landscape documented in Section 2, is increasingly the difference between a skill that runs and one that is blocked.

The signature also functions as portable reputation. A GPG fingerprint is not tied to a platform. An author who signs skills today builds a verifiable identity that is recognizable across any agent runtime that supports SkillSeal verification — the same key, the same fingerprint, the same trust relationship.

12.2 Security Reviewers

The attestation system creates a distinct role: the security reviewer. A reviewer examines a skill's contents, signs a statement binding their identity to the exact artifact digests, and publishes the attestation independently. The skill author is not involved and cannot prevent or control the review.

This role does not exist in the current agent ecosystem. No mechanism allows a security-conscious individual or team to say "I have reviewed this skill and vouch for its contents" in a way that agents can verify mechanically. Attestations make this possible.

A reviewer who builds a track record of careful attestations becomes a trust anchor. Other users add the reviewer's fingerprint to their local trust store, and every skill the reviewer has attested becomes executable without further intervention. This is the same trust model as a package maintainer in a Linux distribution — except it operates without a central distribution authority.

The reviewer convention — hosting attestations in a public `skillseal-attestations` repository on GitHub — means reviews are transparent, auditable, and versioned. Anyone can inspect what a reviewer has attested, when, and for which versions.

12.3 Open Source Communities

When multiple authors sign and multiple reviewers attest, a web of trust emerges organically. No central registry coordinates this. Each user's trust store reflects their own trust decisions: which authors they trust directly, which reviewers they rely on for skills from unknown authors.

This mirrors how trust actually works in open source. Developers trust certain maintainers, certain security researchers, certain organizations — not because a central authority told them to, but because reputation accumulates through consistent, verifiable behavior. SkillSeal makes this implicit trust explicit and mechanically enforceable.

The decentralized structure also means that different communities can have different trust graphs. A security-focused community might require attestations from specific reviewers. A research community might trust a smaller set of prolific authors directly. Neither community needs to convince the other to change their trust model.

12.4 Enterprise Environments

SkillSeal does not include enterprise-specific features. It does not need to.

The composable design principle means that SkillSeal's artifact-layer signatures are consumable by any system that can parse JSON and invoke GPG. An enterprise that wants centralized enforcement can build a proxy that runs `skillseal verify` at the network boundary. An organization that wants managed trust can distribute a signed trust store file that agents merge with their local configuration. A compliance team that wants audit logging can collect verification results from hook output.

None of these require changes to SkillSeal itself. The protocol is the same. The signature format is the same. The trust store schema is the same. Enterprise tooling is a consumer of the standard, not a feature of it.

This is the same relationship that TLS certificates have with enterprise certificate management. The certificate format does not change for enterprise use. The enterprise builds policy, distribution, and monitoring on top of a standard that works identically for an individual and a Fortune 500.

13 Security Analysis

13.1 Timing-Safe Comparisons

All hash and fingerprint comparisons in SkillSeal use constant-time comparison via `crypto.timingSafeEqual()`:

```
import { timingSafeEqual } from "node:crypto";

function safeCompare(a: string, b: string): boolean {
  const maxLen = Math.max(a.length, b.length);
  const bufA = Buffer.alloc(maxLen);
  const bufB = Buffer.alloc(maxLen);
  bufA.write(a);
  bufB.write(b);
  return timingSafeEqual(bufA, bufB) && a.length === b.length;
}
```

Both inputs are padded to equal length before comparison, ensuring `timingSafeEqual()` always receives same-length buffers. The length check is performed after the constant-time comparison to avoid leaking length information through early return. While timing attacks against local hash comparisons have limited practical impact, this eliminates the vector entirely — particularly relevant for enterprise proxy deployments where comparisons may occur over a network boundary.

13.2 Input Validation

- **GitHub usernames** are validated against `^[a-zA-Z0-9_-]+$` before URL construction
- **TRUST.json** is schema-validated after parsing (required fields, expected types)
- **Key fetch responses** are checked for PGP public key block headers, with size limits and timeouts
- **File paths** are resolved to absolute paths to prevent argument injection against GPG (paths starting with `--`)

13.3 Process Isolation

Verification uses `Bun.spawn()` with array-style argument passing (exec-style, not shell). This prevents shell injection regardless of file path content. The temporary GPG keyring provides further isolation — verification never touches the user's default keyring.

13.4 Known Limitations

Key revocation is trust-store scoped. Trust bundles support a `revoked_fingerprints` list that prevents verification of signatures from compromised keys. However, revocation is per-trust-store — there is no global revocation broadcast. If an author's key is compromised, each trust store must independently add the fingerprint to its revocation list. GPG-level revocation is also detected (see GPG Revocation Detection above).

GitHub as single key source. Key discovery depends on GitHub's availability and integrity. Fingerprint pinning mitigates key substitution but not a complete GitHub outage (verification fails, which is the fail-closed behavior).

Minimal YAML parser. The frontmatter parser handles single-line `key: value` pairs. Complex YAML constructs (multi-line values, anchors, aliases) are not supported. This is a deliberate constraint — the frontmatter format is restricted.

Advisory locking for trust store writes. Trust store write operations use advisory file locking (`wx` exclusive-create flag) to prevent concurrent modification. This is cooperative — processes that bypass SkillSeal's APIs can still race. For read-only verification this is not a concern.

13.5 Key Cache

Verification fetches public keys from GitHub on every run. To support offline and air-gapped environments, SkillSeal caches fetched keys locally at `~/.skillseal/key-cache/`. Keys are cached on successful fetch and served from cache when GitHub is unavailable. There is no TTL — cached keys persist until manually cleared.

Setting `"offline": true` in `~/.skillseal/config.json` reduces the fetch timeout to 1 second, falling back to cache immediately. This enables fully offline verification when keys have been previously cached.

13.6 GPG Revocation Detection

Before verifying a GPG signature, SkillSeal checks if the signing key has been revoked. It inspects the output of `gpg --list-keys --with-colons` for a `pub:r` record (revoked public key). If the key is revoked, verification fails immediately — even if the cached signature would otherwise be valid.

This closes a gap in the previous model: a compromised key that was revoked via GPG's standard mechanism would still produce valid signatures in SkillSeal. Now, revocation propagates through GPG's existing infrastructure.

13.7 Attestation Liveness Probe

For attestations discovered locally (in the `ATTESTATIONS/` directory), SkillSeal performs a HEAD request to the reviewer's expected remote repository to check if the attestation still exists. If the remote returns 404, the attestation is treated as stale with a warning that it may have been withdrawn.

This provides a soft revocation mechanism for attestations — a reviewer who discovers a problem can delete their attestation from their repository, and SkillSeal will detect the withdrawal on next verification.

Network errors do not block verification (the probe is best-effort). In offline mode, the probe is skipped entirely.

14 Future Work

Extended key revocation. GPG revocation detection is implemented (v0.2.5). Future work includes SSH key revocation, cross-platform revocation propagation, and a revocation timeline that detects signatures made after a key was compromised.

Automated scanning attestations. Integration with static analysis tools that can automatically generate `automated-scan` attestation bundles. An LLM agent reviews skill instructions for suspicious patterns and signs its findings.

Hardware key support. Integration with YubiKey, Nitrokey, and other hardware security modules for signing operations. Particularly relevant for enterprise environments where keys must not exist as files on disk.

Additional signing providers. The pluggable provider architecture enables future support for Sigstore (keyless OIDC-based signing), age encryption keys, or other emerging standards. Providers register via `registerProvider()` and implement the `SigningProvider` interface.

15 Conclusion

LLM agents execute skill packages and plugins as a core workflow. These artifacts function as installers with full system privileges — yet no standard mechanism exists to verify their provenance or integrity.

SkillSeal provides a multi-key cryptographic signing framework that is lightweight enough for individual authors, self-bootstrapping from a single trust anchor, decentralized without centralized infrastructure, extensible through a pluggable provider architecture, and enforceable at the point of execution.

The threat is real and documented. The gap in the current tooling landscape is specific and well-defined. The solution composes cleanly with existing security infrastructure rather than attempting to replace it.

Markdown is an installer. Installers should be signed.

16 Appendix A: Specifications

The following specifications define the formats used by SkillSeal:

- **Signature Format** (`spec/signature-format.md`): Detached GPG signature format, manifest schema, YAML frontmatter fields, TRUST.json schema, and the signing/verification flows.
- **Trust Store Format** (`spec/trust-store-format.md`): Trust store schema, integrity protection, policy scenarios and actions, trust decision flow, and CLI management.
- **Attestation Format** (`spec/attestation-format.md`): Attestation bundle format, canonical JSON serialization, signing and verification processes, discovery mechanisms, and staleness semantics.

All specifications are versioned at `0.2.5` and included in the SkillSeal repository.

17 Appendix B: CLI Reference

```
skillseal <command> [options]

Commands:
  sign <dir>          Sign a skill or plugin with all configured keys
  verify <dir>        Verify signature, manifest, and trust policy
  sign-all <dir>      Sign all skills and plugins in a directory tree
  attest <dir>        Create a multi-key attestation bundle
  init <dir>          Scaffold a new skill package
  trust add           Add an author or reviewer to the trust store
  trust remove        Remove an entity from the trust store
  trust add-key       Add a key to an existing trust store entity
  trust remove-key    Remove a key from an entity
  trust list          List all trusted entities and their keys
  trust set-policy    Change a policy action
  trust override add   Add a per-skill destatement override
  trust override remove Remove a per-skill override
  trust override list  List all overrides
  trust bundle add     Subscribe to a trust bundle
  trust bundle update  Fetch and merge bundle updates
  trust bundle list    List bundle subscriptions
  cache-clear         Clear all provider caches

Verify options:
  --attestation <path> Path or URL to an attestation bundle
  --human             Human-readable output

Attest options:
  --scope <scope>     Review scope (full-review, security-audit,
```

```
                      automated-scan, functional-review)
  --statement <text>  Review statement
  --output <path>     Output file path
  --reject            Create a destatement (negative attestation)
  --human             Human-readable output

Trust options:
  add <github> <fp>       Add trusted author (--reviewer for reviewer)
  remove <github>         Remove from trust store
  add-key <github> <fp>   Add a key to an existing entity
  remove-key <github> <fp> Remove a key from an entity
  list                    Show all trusted entities
  set-policy <s> <a>      Set policy for scenario to action
  override add <skill> --despite <gh> [--reason "..."]
  override remove <skill> --despite <gh>
  override list
  bundle add <org/repo>
  bundle update
  bundle list

Configuration (~/.skillseal/config.json):
  {
    "github": "username",
    "author": "Your Name",
    "keys": [
      { "type": "gpg", "fingerprint": "..." },
      { "type": "ssh", "fingerprint": "SHA256:...", "key_path": "~/.ssh/key" }
    ]
  }
```